Benchmark Averages Hide the Failures That Matter: Quantizing ESM-2 for Protein Variant-Effect Prediction

Qing Shao
Department of Chemical and Materials Engineering
University of Kentucky, Lexington, KY 40506

August 9, 2026

Abstract

We benchmark six numerical precision configurations for ESM-2 protein language models across three axes — throughput, memory footprint, and predictive accuracy — on two workloads with sharply different characteristics: bulk embedding extraction and deep mutational scanning (DMS) variant-effect scoring. Accuracy is evaluated on the complete ProteinGym substitution benchmark — 201 assays, 2.41M variants — at **three model scales spanning 650M to 15B parameters**, with a paired bootstrap clustered on protein. Three findings follow, and each contradicts a common practice. First, **benchmark averages conceal the failure that decides deployability**: no configuration shifts mean correlation by more than 0.007 at any scale, yet INT8 dynamic quantization — indistinguishable from fp32 on that mean at 3B ($p = 0.34$) — takes a single assay from $\rho = 0.591$ to 0.223. Selection must be made on worst-case, not mean, behaviour. Second, **fidelity measured against fp32 bounds risk but cannot rank quality**: over 3015 assay/configuration pairs it predicts the *magnitude* of ground-truth change ($r = 0.56$–0.81) but not its *direction*, and INT4 weight-only quantization is significantly harmful at 650M, significantly *beneficial* at 3B, and indistinguishable from fp32 at 15B — a reversal with no monotone trend to extrapolate. Third, **the configuration ranking is itself scale-dependent**: FP8 dynamic quantization runs at 0.30× bf16 speed for DMS at 650M and 0.97× at 15B while halving memory, so the workload conflict that dominates at small scale dissolves at large scale. We also report four measurement artifacts encountered during this study, three of which inverted the result they were meant to measure.

Keywords: protein language models; ESM-2; post-training quantization; variant-effect prediction; ProteinGym; benchmark methodology; FP8; INT8

1 Introduction

Quantization is usually presented as a straightforward trade: reduce numerical precision, gain speed and memory, lose a little accuracy. This framing is inherited from large autoregressive language models, where it is broadly correct. We set out to test whether it transfers to ESM-2, a family of protein language models, and found that essentially every part of it requires qualification.

ESM-2 is a BERT-style bidirectional *encoder*. It performs a single forward pass with no KV cache and no autoregressive decode. This places it in a **compute-bound** regime rather than the memory-bandwidth-bound regime that governs LLM inference at batch size 1. The distinction is not academic: the dominant open-source quantization toolchains (GPTQ, AWQ, bitsandbytes NF4,

GGUF) are all *weight-only*. They store INT4/INT8 weights and dequantize to bf16 immediately before an ordinary bf16 matrix multiply. When the bottleneck is memory bandwidth, this is a large win. When the bottleneck is arithmetic, as it is here, it adds work to an already-saturated kernel.

This paper addresses three questions:

1. What do the available quantization configurations actually cost and deliver on the three axes of speed, memory, and accuracy?
2. Does the answer depend on the workload? (It does, and the two workloads we study prefer opposite configurations.)
3. Do the fidelity metrics conventionally used to validate quantization predict real-world predictive accuracy? (Only partially, and in a way that matters.)

Contributions

- An evaluation of six precision configurations for ESM-2 on the **complete ProteinGym substitution benchmark** — 201 assays, 2.41M variants, 40,775 masked positions — at **three model scales** (650M, 3B, 15B), rather than on drift from the full-precision model or on a handful of assays. 3618 assay/configuration runs in total.
- The finding that **benchmark-mean accuracy is nearly uninformative for deployment**. No configuration shifts mean correlation by more than 0.007 at any scale, while one that passes the mean test at $p = 0.34$ takes a single assay from $\rho = 0.591$ to 0.223. We argue that worst-case single-assay drift, not the mean, is the selection criterion, and give a label-free protocol for measuring it on a target (Algorithm 3).
- A quantitative characterisation of what drift-from-fp32 metrics buy: over 3015 assay/configuration pairs they predict the magnitude of ground-truth change ($r = 0.56$–0.81) but not its direction. The direction does not merely vary — INT4 quantization is significantly harmful at 650M, significantly beneficial at 3B, and null at 15B, so there is no trend in scale to extrapolate either.
- The observation that **the configuration ranking is scale-dependent**. The two common ESM-2 workloads prefer opposite configurations at 650M and 3B, but FP8 dynamic quantization closes from 0.30× to 0.97× bf16 DMS speed as scale grows, so the conflict dissolves by 15B. A recommendation derived at one scale does not transfer.
- Evidence that **scale does not buy variant-effect accuracy**: fp32 mean ρ falls monotonically $0.4459 \to 0.4398 \to 0.4323$ across a 23-fold parameter increase, with no pairwise difference reaching significance.
- A documented account of **four measurement artifacts**, three of which inverted the quantity being measured while producing plausible, monotonic result tables — offered as reusable hazards rather than as local mistakes.
- Open code, per-variant predictions for all 18 model/configuration combinations, and per-assay statistics (Section 9).

2 Background and Related Work

Protein language models. ESM-2 [7] is a family of transformer encoders trained with masked language modelling on UniRef, following ESM-1b [12]. Architecturally these are BERT-style

bidirectional encoders [2]: a single forward pass, no KV cache, no autoregressive decode. Meier et al. [9] established that such models predict variant effects zero-shot via *masked marginals* — masking a mutated position and comparing the log-probabilities of mutant and wild-type residues — which is the scoring protocol used throughout this work.

Variant-effect benchmarks. ProteinGym [11] aggregates deep mutational scanning assays into a standard benchmark; the v1 substitution set used here contains 217 assays spanning 2.47M variants and five functional categories, including the mega-scale stability measurements of Tsuboyama et al. [15]. Reporting mean Spearman correlation across assays is the established summary statistic, and one of our contributions is to show what that statistic conceals.

Post-training quantization. The dominant open-source toolchains are weight-only: GPTQ [6] and AWQ [8] store INT4 weights and dequantize before a bf16 matmul, while LLM.int8() [3] introduced mixed-precision decomposition to handle activation outliers. These were designed for autoregressive decoding, where batch-1 inference is memory-bandwidth-bound and dequantization is effectively free. Weight–activation (W8A8) schemes and the FP8 E4M3/E5M2 formats [10] instead reduce arithmetic cost, which is what a compute-bound encoder requires. We use `torchao` [14] for all configurations and PyTorch 2's compiler stack [1], whose Inductor backend emits Triton [13] kernels; Section 4.1 shows that the compile mode, not the quantization scheme, determines whether W8A8 is viable at all. Quantization is also load-bearing for parameter-efficient fine-tuning [4], a setting we do not evaluate.

Positioning. Quantization studies for protein language models are comparatively rare, and where accuracy is reported it is typically drift against the full-precision model on a small probe set. Our contribution is to replace that proxy with the full downstream benchmark at two model scales, and to show that the proxy is a valid risk bound but an invalid quality ranking.

3 Experimental Design

3.1 Configurations

Six configurations were evaluated, all applied via `torchao` to the encoder `Linear` layers only:

Table 1: Quantization configurations. Weight-only variants target memory; the dynamic (W8A8) variants also quantize activations and can therefore use low-precision tensor cores for the matrix multiply itself.

Configuration	Class	Description
`fp32`	baseline	Reference for all fidelity measurements.
`bf16`	baseline	The practical production default.
`int8_wo`	weight-only	INT8 weights, bf16 compute. Memory-oriented.
`int4_wo`	weight-only	INT4 weights, bf16 compute. Smallest footprint.
`int8_dyn`	W8A8	INT8 weights *and* dynamically quantized activations; uses INT8 tensor cores.
`fp8_dyn`	W8A8	FP8 (E4M3) weights and activations. Requires compute capability ≥ 8.9; H200 only.

3.2 Workloads

Bulk embedding extraction. Large length-bucketed batches under a token budget, measured in residues per second. This is the compute-bound regime.

DMS variant-effect scoring. The standard ESM masked-marginals protocol: for each mutated position, mask it and record $\log p(\text{mut}) - \log p(\text{wt})$ from the resulting distribution. Cost scales with the *number of mutated positions*, not with the number of variants, and the masked batch is narrow. This is the latency-bound regime.

Algorithm 1 states the scoring procedure as implemented. Two optimizations, both absent from typical reference implementations, are marked: only positions carrying a variant are masked (line 5), and the masked copies are batched rather than run one at a time (line 6). Together they are worth one to two orders of magnitude on a real assay — the median ProteinGym assay mutates 96 positions out of $L = 222$ — and they are applied identically to every configuration, so they change the absolute cost of the benchmark without affecting any comparison within it.

Algorithm 1 Masked-marginals variant scoring, as benchmarked. Cost is $\lceil |P|/B \rceil$ forward passes and is independent of $|V|$, so an assay with 16,000 variants over 900 positions costs the same as one with 900.

Require: wild-type sequence s, variants V, model f_θ, batch size B
Ensure: score $\sigma(v)$ for every $v \in V$

1: $M \leftarrow \{\, \text{parse}(v) : v \in V \,\}$ ▷ each v is a set of $(\text{wt}, \text{pos}, \text{mut})$ triples
2: **for** each $(a, i, b) \in \bigcup M$ **do**
3: **assert** $s_i = a$ ▷ indexing check; a mismatch is silent and unrecoverable later
4: **end for**
5: $P \leftarrow \{\, i : (a, i, b) \in \bigcup M \,\}$ ▷ **only** variant-carrying positions, not all L
6: **for** each chunk $C \subseteq P$ with $|C| = B$ **do**
7: $X \leftarrow |C|$ copies of tokenize(s); set $X_{r,\,c_r} \leftarrow$ `[MASK]` ▷ one masked position per row
8: $Z \leftarrow f_\theta(X)$ ▷ single batched forward pass
9: **for** each r, with $c_r \in C$ **do**
10: $\ell_{c_r} \leftarrow \log \text{softmax}(Z_{r,\,c_r}$ in fp32) ▷ fp32 softmax regardless of compute precision
11: **end for**
12: **end for**
13: **for** each $v \in V$ **do**
14: $\sigma(v) \leftarrow \sum_{(a,i,b) \in M_v} \big(\ell_i[b] - \ell_i[a]\big)$ ▷ difference of near-equal terms: noise does not cancel
15: **end for**

The final line is why DMS ρ is the most sensitive fidelity metric we record. The score is a difference of two nearly equal log-probabilities, so quantization error in ℓ_i does not cancel between the two terms; it is amplified relative to the signal.

3.3 Metrics

Three fidelity metrics were recorded, deliberately ordered by sensitivity:

1. **Embedding cosine similarity** (mean-pooled, excluding special tokens and padding). Forgiving: error averages out along the sequence.

2. **Logit KL divergence**, per residue. Moderately sensitive.

3. **DMS Spearman ρ against fp32.** Most sensitive, because the score is a *difference of two near-equal log-probabilities*, so quantization noise does not cancel and is amplified relative to signal.

All three are drift-from-fp32 measures. They quantify how far a configuration moves the answer, not whether it moves toward or away from the truth. To measure the latter, we added a fourth metric requiring real experimental data: **Spearman ρ against wet-lab DMS measurements**, from ProteinGym.

3.4 Real assay data

Three ProteinGym substitution assays were selected to span the sequence-length range, since peak memory during masked-marginals scoring grows with L:

Table 2: ProteinGym assays used. All 19,557 single-substitution variants had their stated wild-type residue verified against the reference `target_seq` before use; all matched. This check is load-bearing — an off-by-one in position indexing does not raise an error, it silently scores the wrong positions and yields a plausible-looking correlation.

Assay	L	Singles	Masked positions	Coverage
`IF1_ECOLI_Kelsic_2016`	72	1,367	72	100%
`BLAT_ECOLX_Stiffler_2015`	286	4,996	263	92%
`HSP82_YEAST_Flynn_2019`	709	13,194	707	100%

3.5 Full benchmark data

The three-assay analysis is superseded by a run over the complete ProteinGym substitution benchmark. All 217 assays were extracted from the `ProteinGym_v1` parquet release and every one passed wild-type validation across 2,465,767 variants. The 201 assays fitting ESM-2's 1022-residue context were scored: 2,413,913 variants over 40,775 masked positions, spanning $L = 37$ to 934 and five function categories.

The masked-position count, not the variant count, sets the cost: masked-marginals scoring performs one forward pass per mutated position, so 2.41M variants are nearly free on top of 40,775 forwards.

An earlier three-assay path combined per-assay CSVs from the `v0.1` release with wild-type sequences from the `v1.0` reference file. The two agree on the 85 assays present in both and diverge elsewhere, including renames. That is acceptable for three hand-picked assays and unsafe at benchmark scale; the `v1` parquet carries `target_seq` inline, so scores and wild-type sequences come from a single release.

3.6 Statistical treatment

Ground-truth correlations are compared using a **paired bootstrap** (2000 resamples). Both the candidate configuration and the fp32 reference are scored on the *same* resample at each iteration, so shared noise cancels and the residual is attributable to the configuration. The pairing is not a refinement: for `int4_wo` the paired interval is 8.4× narrower than the unpaired one, because per-assay ρ spans 0.0–0.9 while the configuration effect is of order 0.006. Unpaired, every configuration reads as noise.

The *resampling unit* differs by scale of claim. For a single assay the unit is the variant, which establishes that a shift is not an artifact of the variant set. For the full benchmark the unit is the assay, since the claim becomes one about assays in general. Because the 201 assays cover only 173 distinct proteins, benchmark-level intervals use a **cluster bootstrap over proteins**: whole proteins are drawn and all their assays retained, so correlated replicates cannot count as independent evidence. Clustered and unclustered intervals agreed to the fourth decimal, which is reported as a measured result rather than assumed.

Intervals are percentile bootstrap [5]. They were checked against a normal approximation ($\bar{d} \pm 1.96\,\mathrm{SE}$) and agree to four decimals on every configuration, confirming the underlying distribution is near-symmetric and the percentile method is not distorting the interval. The procedure was validated on synthetic data with a known answer: a negligible perturbation was correctly classified as noise, a genuine improvement as real.

Algorithm 2 states the benchmark-level test. The pairing is line 5: one resample index is applied to the candidate and to the reference *together*. Without it the interval widens by 8.4× for `int4_wo`, because between-assay variance in ρ (spanning 0.0 to 0.9) is two orders of magnitude larger than the configuration effect (≈ 0.006), and every configuration is then reported as noise.

Algorithm 2 Paired cluster bootstrap for Δ(mean ρ) against the fp32 reference. Setting $\mathcal{G}$ to singletons recovers the ordinary assay-level bootstrap; we report both.

Require: per-assay correlations $\rho_a^c, \rho_a^{\mathrm{ref}}$ for assays $a = 1{:}n$; protein clusters $\mathcal{G}$ partitioning the assays; $R = 2000$
Ensure: point estimate Δ, 95% interval, two-sided p
1: $\Delta \leftarrow \frac{1}{n}\sum_a \rho_a^c - \frac{1}{n}\sum_a \rho_a^{\mathrm{ref}}$
2: **for** $r = 1$ **to** R **do**
3: draw $|\mathcal{G}|$ clusters from $\mathcal{G}$ with replacement
4: $S_r \leftarrow$ concatenation of all assays in the drawn clusters ▷ $|S_r|$ varies between replicates
5: $d_r \leftarrow \frac{1}{|S_r|}\sum_{a \in S_r}\left(\rho_a^c - \rho_a^{\mathrm{ref}}\right)$ ▷ **same** S_r indexes both arms
6: **end for**
7: $[\,\ell, u\,] \leftarrow$ 2.5th and 97.5th percentiles of $\{d_r\}$
8: $p \leftarrow 2\min\left(\frac{1}{R}|\{d_r \le 0\}|,\ \frac{1}{R}|\{d_r \ge 0\}|\right)$ ▷ floor $2/R = 0.001$; report smaller values as $p < 0.002$
9: **return** Δ, $[\ell, u]$, p; call the effect real iff $0 \notin [\ell, u]$

3.7 Hardware and software environment

Table 3: Hardware and software environment. The glibc 2.17 ceiling is what forces torch 2.6.0 and rules out the prebuilt bitsandbytes GPU wheels, so NF4 was unavailable and is absent from the configuration list.

Component	Detail
GPU (primary)	NVIDIA H200, 143 GB, sm_90 (FP8 tensor cores)
GPU (secondary)	NVIDIA A100, 80 GB, sm_80 (no FP8)
Driver	550.54.14
PyTorch	2.6.0+cu124
Attention	SDPA
Compile mode	`max-autotune-no-cudagraphs`

The cluster runs CentOS 7.6 with glibc 2.17 on every node, including the H200 nodes, which forced several choices. PyTorch moved to `manylinux_2_28` at version 2.7, making **2.6.0 the newest**

installable release. `bitsandbytes` GPU wheels require a newer glibc, so only the CPU-only 0.42.0 installs and its NF4 path is unavailable — 4-bit quantization therefore goes through `torchao`. The system GCC (4.8.5) cannot build Triton's helper module, so `torch.compile` fails until `CC`/`CXX` are pointed at GCC 12.1.0. Finally, `$HOME` and `/project` are at quota, and the Triton, Inductor, HuggingFace, and pip caches all default there; all were relocated to `/scratch`. `torchao` nonetheless works because its INT8/FP8 paths dispatch to `torch._int_mm` and `torch._scaled_mm`, which live inside PyTorch and support sm_90.

4 Results

4.1 Compile mode determines whether W8A8 is viable at all

The single most consequential configuration choice is not the quantization scheme but the `torch.compile` mode. Measured on a single ESM2-3B-shaped feed-forward block, $2560 \to 10240 \to 2560$, at batch 32×512 on an A100:

Table 4: Only `max-autotune` allows Inductor to autotune the INT8 Triton matmuls, which is where the entire W8A8 benefit resides.

Variant	Time (ms)
bf16 eager	7.91
bf16 compiled	7.91
`int8_dyn` eager	33.35
`int8_dyn` compiled, `dynamic=True`	11.73
`int8_dyn` compiled, default mode	11.48
`int8_dyn` `mode="max-autotune"`	**5.43**

Measured in eager mode, W8A8 appears 4.2× *slower* than bf16 and would be discarded. Measured under `max-autotune` it is 1.46× faster. `dynamic=True` was avoided because it emits shape-generic kernels that surrender most of the gain; static shapes are kept manageable by padding to a fixed multiple of 128. `torch._dynamo.explain` confirms **zero graph breaks** for both bf16 and `int8_dyn`, establishing this as a kernel-selection effect rather than a Dynamo fallback.

4.2 Bulk embedding extraction

Table 5: ESM2-3B, H200, compiled with `max-autotune-no-cudagraphs`, SDPA attention, 64 sequences of length 512–1024 under a 16,384-token budget. Reproduced across four independent jobs: bf16 mean 56,642 residues/s, range 55,941–57,422 (±1.3%); fp32 mean 6,742, range 6,687–6,814.

Config	Weights (GB)	Peak (GB)	Residues/s	vs bf16	Emb. cos	Logit KL	DMS ρ_{fp32}
`fp32` (ref)	10.72	25.37	6,750	0.12×	—	—	—
`bf16`	5.29	12.63	57,057	1.00×	0.999949	1.17e–04	0.9998
`int8_wo`	2.66	8.48	54,816	0.96×	0.999959	1.05e–04	0.9998
`int8_dyn`	2.65	**7.07**	59,996	1.05×	0.989592	1.40e–02	0.9928
`fp8_dyn`	2.65	7.24	**65,376**	**1.15**×	0.999920	4.60e–04	0.9988
`int4_wo`	**1.61**	7.69	3,813	0.07×	0.998904	1.89e–03	0.9975

Three observations. First, the **fp32 → bf16 step is worth** 8.4× — larger than everything quantization adds on top of it, and the single biggest lever available. Second, `fp8_dyn` dominates `int8_dyn`: it is faster (1.15× vs 1.05×) *and* dramatically more accurate (embedding cosine 0.999920 vs 0.989592; logit KL 4.6×10^{-4} vs 1.4×10^{-2}), surrendering only 0.17 GB of peak memory. FP8's E4M3 format has far greater dynamic range than INT8 at the same bit width, which matters because transformer activations contain outliers. Third, `int4_wo` is a trap for inference: 0.07× throughput and *worse* peak memory (7.69 GB) than `fp8_dyn` (7.24 GB) despite 40% smaller weights, because the dequantization path allocates transient buffers. Its value lies elsewhere, as a QLoRA base.

Separately, length-bucketed batching reduced padding waste from 25.0% (naive fixed-size batching) to **0.0%**. This is free and costs no accuracy — a larger effect than several of the quantization differences in the table.

4.3 DMS scoring: the opposite conclusion

Table 6: DMS scoring throughput, ESM2-3B, uncompiled. bf16 wins at every problem size; quantization is pure overhead in this regime.

Config	Variants/s		
	IF1 (L=72)	BLAT (L=286)	HSP82 (L=709)
`bf16`	**6,979**	**3,573**	**1,357**
`int8_wo`	4,830	2,511	1,066
`fp8_dyn`	1,813	1,796	1,103
`fp32`	1,460	447	187
`int8_dyn`	410	382	299
`int4_wo`	1,045	282	112

bf16 is fastest at every assay size, and `int8_dyn` — the second-fastest configuration for bulk extraction — is up to 17× slower here. The masked batch (n_positions × L) is too small to contain a matrix multiply large enough for low precision to pay for its quantize/dequantize overhead.

Notably, `fp8_dyn` closes on bf16 as the assay grows: 0.26×, 0.50×, 0.81× of bf16 throughput at $L = 72, 286, 709$ respectively. Its overhead is fixed per call while real work scales, so at $L = 709$ it costs 20% throughput for 43% less memory — a defensible trade that is indefensible at $L = 72$.

4.4 Memory during DMS scoring

Table 7: Peak device memory during DMS scoring. Weights account for approximately 90% of peak even at $L = 709$ (bf16: 5.29 GB weights against 0.59 GB activations).

Config	Weights (GB)	Peak L=72	Peak L=286	Peak L=709
`fp32`	10.72	10.87	11.19	11.85
`bf16`	5.29	5.38	5.55	5.88
`int8_wo`	2.66	2.77	2.92	3.25
`int8_dyn`	2.65	2.87	2.91	3.36
`fp8_dyn`	2.65	2.76	2.96	3.35
`int4_wo`	**1.61**	**1.70**	**1.87**	**2.20**

This is the reverse of the bulk-extraction situation. Masked-marginals scoring keeps the batch narrow, so the $O(L^2)$ attention term never dominates and weights remain roughly 90% of peak

memory even at $L = 709$. Quantization therefore addresses almost all of DMS peak memory, and `int4_wo` gives the smallest footprint at every length — 2.7× below bf16.

4.5 Ground-truth accuracy on ProteinGym

Table 8: Spearman correlation against experimental measurements, ESM2-3B. Δ is the change from fp32, with a 2000-sample paired bootstrap over variants. "Real" indicates the 95% confidence interval excludes zero.

Assay	Config	ρ_{expt}	Δ vs fp32	95% CI of Δ	Verdict
IF1	`fp32`	0.5561	(ref)		
(n=1367)	`bf16`	0.5543	−0.0018	[−0.0027, −0.0009]	**real**
	`int8_wo`	0.5570	+0.0009	[−0.0003, +0.0021]	noise
	`int8_dyn`	0.5504	−0.0057	[−0.0124, +0.0013]	noise
	`fp8_dyn`	0.5584	+0.0023	[−0.0022, +0.0067]	noise
	`int4_wo`	0.5394	**−0.0167**	[−0.0263, −0.0074]	**real**
BLAT	`fp32`	0.5893	(ref)		
(n=4996)	`bf16`	0.5896	+0.0003	[−0.0003, +0.0009]	noise
	`int8_wo`	0.5913	+0.0019	[+0.0011, +0.0029]	**real**
	`int8_dyn`	0.6160	**+0.0266**	[+0.0219, +0.0315]	**real**
	`fp8_dyn`	0.6088	**+0.0194**	[+0.0161, +0.0229]	**real**
	`int4_wo`	0.6388	**+0.0495**	[+0.0433, +0.0561]	**real**
HSP82	`fp32`	0.2931	(ref)		
(n=13194)	`bf16`	0.2933	+0.0002	[−0.0002, +0.0005]	noise
	`int8_wo`	0.2933	+0.0002	[−0.0002, +0.0007]	noise
	`int8_dyn`	0.2937	+0.0006	[−0.0020, +0.0028]	noise
	`fp8_dyn`	0.2944	+0.0012	[−0.0003, +0.0027]	noise
	`int4_wo`	0.2904	−0.0027	[−0.0057, +0.0002]	noise

Two findings emerge, and the second qualifies the first.

Quantization does not systematically degrade variant-effect accuracy. Of the 15 assay/configuration pairs, 11 shifts are upward. Most are statistically indistinguishable from zero. Strikingly, `int4_wo` — the worst configuration on every fidelity metric — produces the *best* agreement with experiment on BLAT (+0.0495, CI [+0.0433, +0.0561]). This survives the bootstrap and is not a resampling artifact.

Fidelity-vs-fp32 predicts the magnitude of the shift, not its sign. Across all 15 pairs, the correlation between fidelity loss $(1 - \rho_{\text{fp32}})$ and the absolute ground-truth shift is $r = 0.87$ (Pearson), $\rho = 0.76$ (Spearman). But `int4_wo` moves +0.0495 on BLAT and −0.0167 on IF1. The direction is a property of the assay, not of the configuration, and is not predictable from any drift-from-fp32 measurement.

The practical consequence is that `bf16` and `int8_wo` shift ground-truth agreement by at most 0.002 on any assay tested — below anything an experiment could resolve — and are therefore safe. `int4_wo` is a ±0.05 gamble on an unknown sign.

This supersedes an earlier conclusion drawn from a synthetic 40-residue mutational scan, which ranked `int8_dyn` ($\rho_{\text{fp32}} = 0.9928$) as risky and `fp8_dyn` (0.9988) as clearly safer. Both numbers

were correct; neither predicted experimental accuracy. On BLAT the "risky" configuration gained +0.027.

We note that the assay itself matters far more than any quantization decision: ρ_{expt} ranges from 0.29 (HSP82) to 0.59 (BLAT) at 3B. Choosing which assay to trust is a 2× effect; choosing a quantization configuration is a 1% effect.

4.6 Model-scale consistency

The same three assays were run on ESM2-650M. The qualitative picture holds: bf16 fastest, `int8_dyn` slowest, ground-truth shifts small. The largest 650M shift was −0.0063 (`int8_dyn` on IF1) against ρ_{fp32} of 0.9954. The larger and more clearly significant shifts at 3B indicate that sensitivity to quantization grows with model size, so 650M results should not be extrapolated to 3B without checking.

An incidental observation deserves note, because it bears directly on how much any of this matters. The fp32 650M model correlates with experiment *better* than the fp32 3B model on two of the three assays:

Table 9: Ground-truth correlation by model scale on the three-assay subset. The 0.14 BLAT gap prompted the inference that model choice dominates precision choice; Section 4.7 shows it does not generalise.

Assay	650M ρ_{expt}	3B ρ_{expt}	Difference
IF1	0.5987	0.5561	−0.0426
BLAT	0.7315	0.5893	−0.1422
HSP82	0.2648	0.2931	+0.0283

The BLAT gap of 0.14 is roughly *three times* the largest effect any quantization configuration produced in this study. Model selection dominates precision selection by a wide margin, and the assumption that the larger checkpoint is the more accurate one does not hold uniformly here.

This inference does not survive Section 4.7. Over 201 assays the two models are statistically indistinguishable ($p = 0.35$), and adding a third scale makes the picture worse for the larger checkpoint rather than better: fp32 mean ρ falls monotonically $0.4459 \to 0.4398 \to 0.4323$ from 650M to 3B to 15B, with no pairwise difference significant ($15\text{B} - 650\text{M} = -0.0137$, $p = 0.14$). The BLAT gap is real for BLAT and does not generalise — the same error, made the same way, as the `int4_wo` result three subsections above. What survives is only the negative claim: there is no evidence that scale buys variant-effect accuracy, and 15B costs 5.4× the compute of 3B for the lowest mean of the three.

4.7 The full benchmark at three model scales

Every accuracy conclusion above rests on three assays and one model. Three assays are enough to establish that an effect exists on a particular assay and not enough to establish anything about assays in general, because the unit that varies — the assay — is held fixed. One model scale is enough to establish that an effect exists for that checkpoint and not enough to know whether it transfers. The whole ProteinGym substitution benchmark was therefore run at three scales: all 217 assays validated, the 201 within ESM-2's 1022-residue context scored, 2,413,913 variants over 40,775 masked positions, six configurations, at 650M, 3B and 15B parameters. 3618 assay/configuration runs, 16.2 GPU-hours.

Two changes to the statistical treatment follow from the larger sample. The resampling unit becomes the **assay** rather than the variant, since the question is now whether a configuration

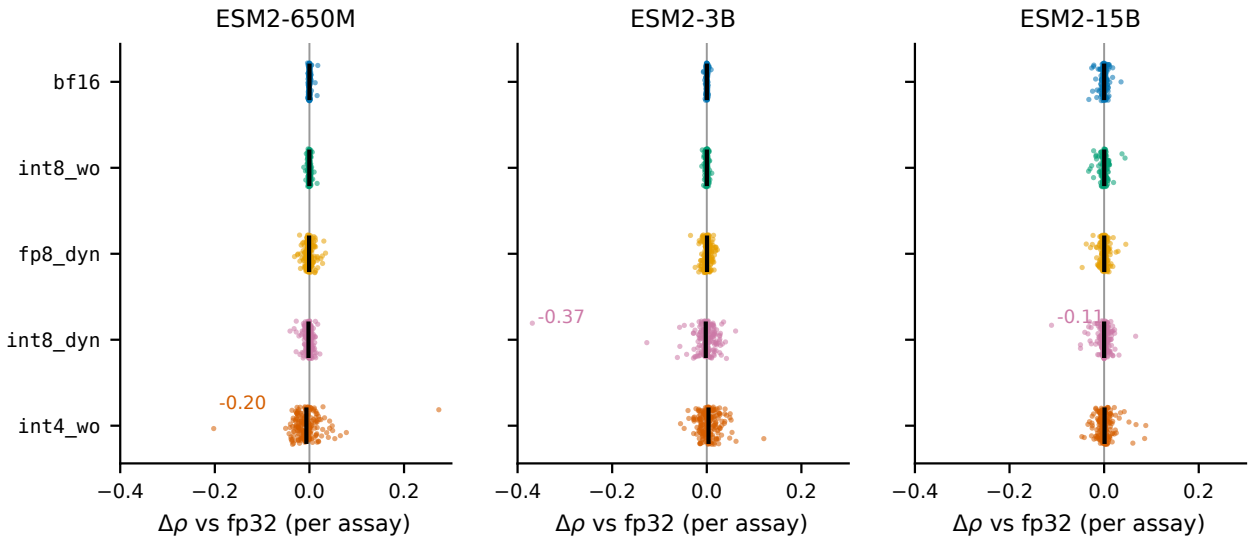

Figure 1: Per-assay change in ground-truth correlation for each configuration, across all 201 ProteinGym assays and all three model scales. Black ticks mark the benchmark mean, which is within 0.007 of zero in every panel. `bf16` and `int8_wo` are tight at every assay and every scale; `int8_dyn` and `int4_wo` have means that are equally unremarkable and tails that reach −0.37 and −0.20. Selection on the mean cannot distinguish these cases; selection on the tail separates them immediately.

degrades prediction in general. And because the 201 assays cover only 173 distinct proteins — `BLAT_ECOLX` appears four times — the reported interval is a **cluster bootstrap over proteins** (Algorithm 2). The two intervals agree to the fourth decimal, so the clustering was immaterial here; it is reported because that had to be measured rather than assumed.

Table 10: Change in mean Spearman against experiment relative to fp32, over 201 assays, at each model scale. Bold marks intervals excluding zero under the protein-clustered paired bootstrap. `int4_wo` is significant in *opposite directions* at 650M and 3B and null at 15B; the effect does not merely vary in size, it has no consistent sign and no monotone trend in scale. fp32 mean ρ is 0.4459, 0.4398 and 0.4323 respectively.

	ESM2-650M		ESM2-3B		ESM2-15B	
Config	Δ	95% CI	Δ	95% CI	Δ	95% CI
`bf16`	+0.0000	[−0.0003, +0.0004]	+0.0002	[−0.0001, +0.0004]	+0.0000	[−0.0009, +0.0011]
`int8_wo`	−0.0002	[−0.0006, +0.0002]	+0.0001	[−0.0002, +0.0005]	+0.0002	[−0.0008, +0.0014]
`fp8_dyn`	−0.0008	[−0.0020, +0.0005]	+0.0005	[−0.0007, +0.0017]	+0.0002	[−0.0010, +0.0015]
`int8_dyn`	**−0.0020**	[−0.0032, −0.0009]	−0.0021	[−0.0072, +0.0017]	−0.0000	[−0.0021, +0.0019]
`int4_wo`	**−0.0064**	[−0.0105, −0.0018]	**+0.0037**	[+0.0007, +0.0068]	+0.0009	[−0.0014, +0.0034]

The benchmark mean is the wrong safety criterion

Every Δ in Table 10 is below 0.007. The per-assay tail is not (Figure 1, Table 11).

`int8_dyn` at 3B is indistinguishable from fp32 on the benchmark mean ($\Delta = -0.0021$, $p = 0.34$) while destroying `UBR5_HUMAN_Tsuboyama_2023_1I2T`, where correlation with experiment falls from 0.591 to 0.223 at $\rho_{\text{fp32}} = 0.393$. A configuration can pass the benchmark average and be unusable on the one protein a given project cares about. This is the practical result of the whole study:

Table 11: Worst single-assay change in ground-truth correlation, with the count of assays moving by more than 0.05 in parentheses, out of 201. The catastrophic tail does not sit with a fixed configuration: it is `int4_wo` at 650M and `int8_dyn` at 3B.

Config	ESM2-650M	ESM2-3B	ESM2-15B
`bf16`	−0.0040 (0)	−0.0072 (0)	−0.0323 (0)
`int8_wo`	−0.0120 (0)	−0.0106 (0)	−0.0329 (0)
`fp8_dyn`	−0.0316 (0)	−0.0344 (0)	−0.0465 (0)
`int8_dyn`	−0.0411 (0)	**−0.3688** (6)	−0.1112 (3)
`int4_wo`	**−0.2024** (6)	−0.0556 (5)	−0.0472 (5)

benchmark-mean significance testing answers a question about populations, and deployment is a question about a specific instance.

Table 11 adds a second warning. The catastrophic tail is not a fixed property of a configuration either — it belongs to `int4_wo` at 650M and to `int8_dyn` at 3B. Knowing which configuration was dangerous at one scale does not tell you which will be dangerous at another.

At 15B the safe configurations degrade and the aggressive one improves

Scale does not move all configurations in the same direction, and the crossover is visible only with a third point (Figure 2c, Table 12). `bf16` and `int8_wo` do not drift measurably from fp32 on *any* of the 402 assay/model pairs at 650M and 3B, and then drift on 13 and 15 assays respectively at 15B, with worst-case rank correlation falling to 0.857 and 0.855. `int4_wo` moves the other way: 182 assays below $\rho_{\text{fp32}} = 0.99$ at 650M, 154 at 3B, 82 at 15B.

Table 12: Fidelity against fp32 by scale: number of assays with $\rho_{\text{fp32}} < 0.99$ (of 201), and the worst single-assay value. The two columns move in opposite directions with scale — the configurations that are safe at small scale are the ones that acquire a tail at 15B, while the most aggressive one loses its tail. Medians stay above 0.99 everywhere, so none of this is visible in a summary statistic.

	assays with $\rho_{\text{fp32}} < 0.99$			worst ρ_{fp32}		
Config	650M	3B	15B	650M	3B	15B
`bf16`	0	0	13	0.9974	0.9905	0.8569
`int8_wo`	0	0	15	0.9963	0.9968	0.8548
`fp8_dyn`	29	18	19	0.9556	0.9770	0.8389
`int8_dyn`	25	108	31	0.8894	0.3930	0.7645
`int4_wo`	182	154	82	0.6707	0.8791	0.8505

A natural reading is that a larger model carries more redundancy, so removing weight precision costs proportionally less, while its activation dynamic range grows until bf16's exponent becomes the binding constraint. We do not have the evidence to assert that mechanism: it would need a per-layer error analysis and an fp16-versus-bf16 comparison, neither of which we ran. What the data supports is narrower and still consequential — **the identity of the risky configuration is not stable across scale**, so a configuration cleared at one scale has not thereby been cleared at another.

The 15B `bf16` degradation is not a long-sequence artifact. Spearman correlation between assay length and ρ_{fp32} is −0.14, and the five worst assays span $L = 222$ to 861.

Table 13: The single worst-affected assay, by configuration and scale. No assay appears twice. Identifying the vulnerable target at one scale does not identify it at another, which is why the screen in Algorithm 3 has to be run against the model actually being deployed.

Scale	Config	Worst assay	$\Delta\rho$
650M	bf16	AICDA_HUMAN_Gajula_2014_3cycles (L=198)	−0.0040
650M	int8_dyn	POLG_PESV_Tsuboyama_2023_2MXD (L=53)	−0.0411
650M	int4_wo	B2L11_HUMAN_Dutta_2010 (L=198)	−0.2024
3B	bf16	ODP2_GEOSE_Tsuboyama_2023_1W4G (L=44)	−0.0072
3B	int8_dyn	UBR5_HUMAN_Tsuboyama_2023_1I2T (L=58)	−0.3688
3B	int4_wo	R1AB_SARS2_Flynn_2022 (L=306)	−0.0556
15B	bf16	I6TAH8_I68A0_Doud_2015 (L=498)	−0.0323
15B	int8_dyn	GFP_AEQVI_Sarkisyan_2016 (L=238)	−0.1112
15B	int4_wo	MBD11_ARATH_Tsuboyama_2023_6ACV (L=66)	−0.0472

Resource envelope at 15B

Table 14: ESM2-15B footprint. Peak memory during masked-marginals scoring is almost entirely weights: the spread across 201 assays spanning $L = 37$ to 934 is under 3 GB for every configuration. Quantization therefore addresses essentially all of the DMS memory cost at this scale, and fp32 alone rules out an 80 GB A100.

Config	Weights (GB)	Peak memory over 201 assays (GB)		
		min	median	max
fp32	56.36	56.51	57.09	59.30
bf16	28.18	28.27	28.57	29.69
int8_wo	14.14	14.39	14.53	15.64
fp8_dyn	14.12	14.22	14.57	15.91
int8_dyn	14.12	14.51	14.90	16.06
int4_wo	7.53	7.62	7.91	9.03

The configuration ranking is scale-dependent

Section 6.1 reports that the two workloads prefer opposite configurations, on the evidence that fp8_dyn wins bulk extraction and loses badly at DMS scoring. That holds at 650M and 3B and stops holding at 15B.

At 15B, fp8_dyn scores the benchmark in 0.381 GPU-hours against bf16's 0.385, with peak memory 14.57 against 28.57 GB and no measurable accuracy cost. It is simply the better choice, which it is not at either smaller scale. The workload conflict is therefore a small-model phenomenon, and a recommendation derived at 650M or 3B does not transfer upward (Figure 2).

What the larger sample changed

The three-assay verdict was backwards, and the correction does not resolve into a trend. Section 4.5 concluded that quantization does not systematically degrade variant-effect accuracy, on the evidence that 11 of 15 shifts were upward and that int4_wo gained +0.0495 on BLAT. At benchmark scale int4_wo is significantly *harmful* at 650M (−0.0064), significantly

Table 15: DMS scoring speed relative to `bf16` over the full 201-assay benchmark, by model scale. Weight-only `int8_wo` holds a constant ratio, as expected for a scheme that adds fixed dequantization work. `fp8_dyn` closes from 0.30× to 0.97×: its overhead is per-call while the useful work grows with L and hidden size, so by 15B the GEMMs are large enough to be compute-bound even in the narrow-batch DMS regime.

	ESM2-650M		ESM2-3B		ESM2-15B	
Config	pos/s	vs bf16	pos/s	vs bf16	pos/s	vs bf16
`bf16`	347.7	1.00×	200.6	1.00×	56.7	1.00×
`int8_wo`	249.0	0.72×	139.3	0.69×	41.0	0.72×
`fp8_dyn`	103.3	0.30×	89.2	0.44×	54.9	**0.97**×
`int8_dyn`	23.8	0.07×	20.1	0.10×	11.3	0.20×
`int4_wo`	55.4	0.16×	17.3	0.09×	3.9	0.07×

beneficial at 3B (+0.0037), and indistinguishable from fp32 at 15B. Two scales would have left this looking like a trend in model size; the third removes that reading. All three measurements are correct and there is nothing to extrapolate from them.

Fidelity predicts magnitude, not direction. Across 3015 assay/configuration pairs, $r(1 - \rho_{\text{fp32}}, |\Delta|)$ is 0.74, 0.81 and 0.56 at 650M, 3B and 15B — consistent with the 0.87 first estimated from 15 pairs. The *signed* correlation is +0.10, −0.58 and −0.40: it does not hold its sign across scales and is therefore useless for prediction. ρ_{fp32} remains the correct cheap screen, bounding how far an answer can move while saying nothing about which way (Figure 3).

Stratified effects do not replicate either. At 650M, `int4_wo`'s cost is monotone in MSA depth — −0.0174, −0.0050, −0.0042 for low, medium and high — which invites the reading that quantization does most harm where the model has least evolutionary signal. That ordering holds at 3B (−0.0011, +0.0007, +0.0097) and breaks at 15B (+0.0011, −0.0009, +0.0033), where every effect is inside the noise band in any case. We report the stratification (Table 16) but draw no conclusion from it: two scales agreeing and a third not is exactly the pattern that the `int4_wo` result should teach us to distrust.

Table 16: `int4_wo` minus fp32 in mean ρ, by ProteinGym function category and by MSA depth, at each scale. The MSA-depth ordering is monotone at 650M and 3B and breaks at 15B; the category ordering does not transfer at all (Binding is the worst stratum at 650M and among the best at 3B). At 15B every effect is inside the noise band of Table 10. We report this for completeness and draw no conclusion from it.

Stratum	ESM2-650M	ESM2-3B	ESM2-15B
Activity	−0.0115	+0.0018	−0.0013
Binding	−0.0184	+0.0049	+0.0004
Expression	+0.0000	−0.0018	+0.0015
OrganismalFitness	−0.0117	−0.0003	−0.0001
Stability	+0.0025	+0.0101	+0.0032
MSA depth: low	−0.0174	−0.0011	+0.0011
MSA depth: medium	−0.0050	+0.0007	−0.0009
MSA depth: high	−0.0042	+0.0097	+0.0033

Speed and memory over the whole benchmark

The DMS recommendation from three assays survives at 650M and 3B and inverts at 15B. Up to 3B, `bf16` eager is fastest at every problem size — 6.2× faster than fp32 end-to-end at 3B — with

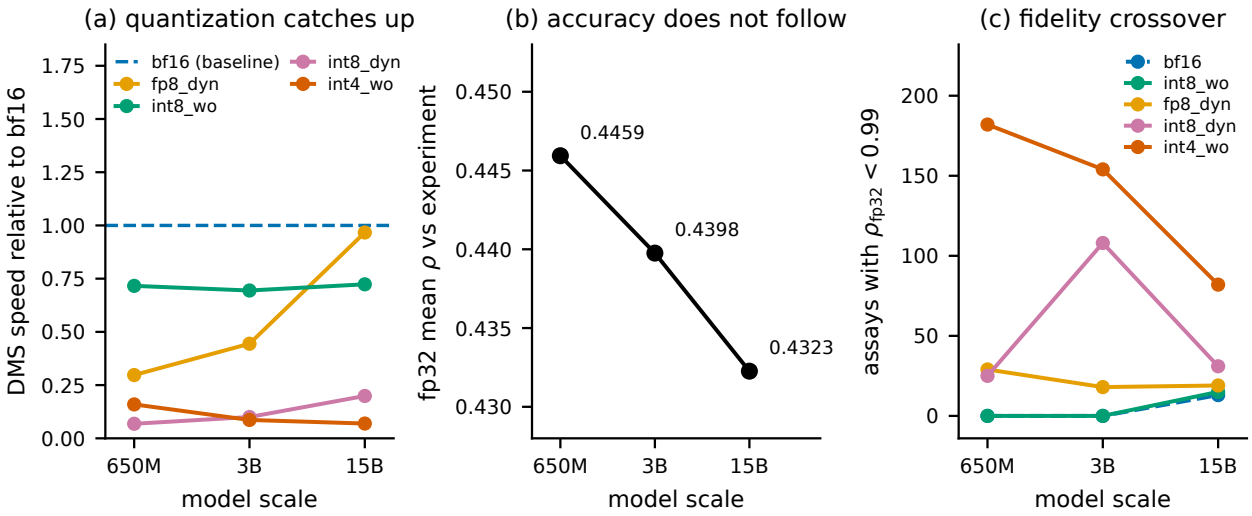

Figure 2: Three effects visible only across scale. **(a)** DMS speed relative to bf16: `fp8_dyn` converges on bf16 while the weight-only and INT4 paths do not, so "quantization is pure overhead for DMS" is scale-bounded. **(b)** fp32 mean Spearman against experiment falls monotonically across a 23-fold parameter increase; no pairwise difference is significant, but there is no evidence that scale buys variant-effect accuracy, and 15B costs 5.4× the compute of 3B for the lowest of the three. **(c)** Number of assays where a configuration drifts from fp32 ($\rho_{\text{fp32}} < 0.99$): `bf16` and `int8_wo` acquire a tail only at 15B, while `int4_wo` sheds one. The safe and the aggressive configurations move in opposite directions.

ground-truth agreement never moving more than 0.018 across 402 assay/model pairs, and `int8_wo` is the memory fallback at roughly 70% of bf16 speed. At 15B, `fp8_dyn` matches bf16 on speed at half the memory and becomes the default. `int8_dyn` and `int4_wo` are not trade-offs for this workload at any scale: they are slower than bf16 *and* carry the two worst tails in Table 11.

5 Reproducibility Pitfalls in Quantization Benchmarking

We document four measurement defects encountered during this study. They are included in the main text, unusually for a paper, because they bear directly on its thesis: three of the four produced clean, monotonic, plausible result tables while measuring the wrong quantity, and each would have supported a confident and wrong recommendation. A reader who accepts our argument that

Table 17: Full 201-assay benchmark by model scale, uncompiled, one H200. Wall-clock is summed scoring time; peak memory is the median across assays. fp32 at 15B needs 57 GB, so the reference configuration alone rules out an 80 GB A100 once activations are included.

	ESM2-650M		ESM2-3B		ESM2-15B	
Config	hours	peak GB	hours	peak GB	hours	peak GB
`fp32`	0.23	2.70	0.75	11.10	3.40	57.09
`bf16`	**0.07**	1.33	**0.12**	5.50	0.39	28.57
`int8_wo`	0.08	0.73	0.15	2.87	0.49	14.53
`fp8_dyn`	0.13	0.75	0.17	2.90	**0.38**	14.57
`int8_dyn`	0.49	0.75	0.67	2.94	1.61	14.90
`int4_wo`	0.39	**0.60**	1.23	**1.82**	5.47	**7.91**

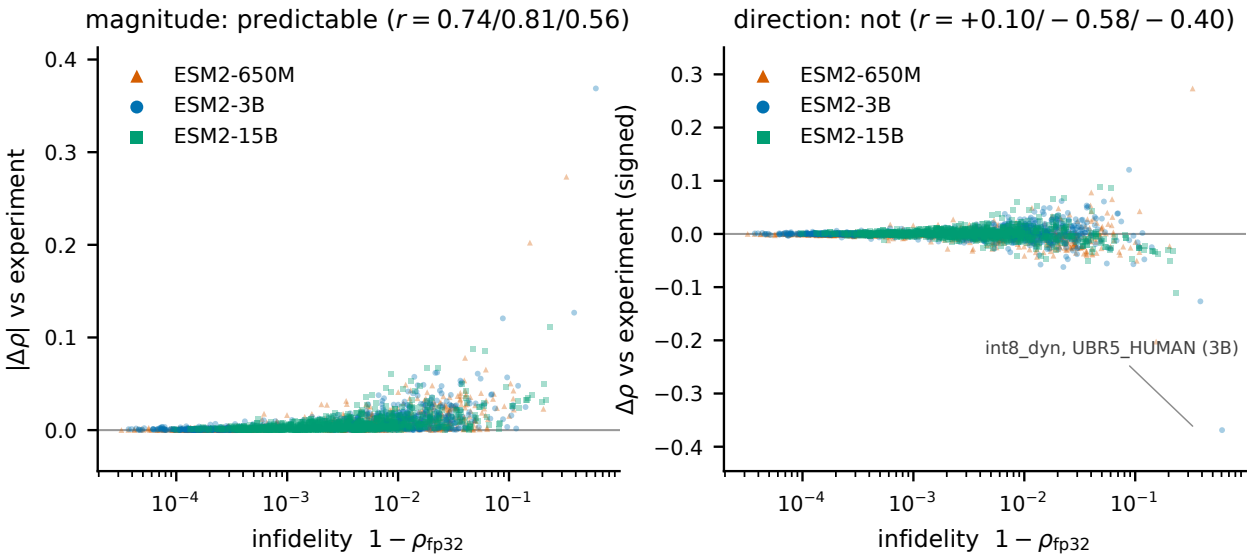

Figure 3: Drift from fp32 against change in ground-truth correlation, over 1005 assay/configuration pairs per model scale. *Left:* infidelity bounds the magnitude of the change — the envelope widens monotonically, and nothing far from fp32 is reliably close to it on the ground truth. *Right:* the same data, signed. The distribution is a symmetric funnel rather than a trend, and the correlation changes sign between scales. A small drift is a valid safety certificate; a large drift is not evidence of a worse model.

benchmark averages can mislead should also want to know how the underlying measurements can mislead. All four are properties of the standard tooling rather than of this codebase, and we expect them to recur in any comparable study.

5.1 Compile cost inside the timed region (three occurrences)

The DMS throughput column initially ranked `fp32` fastest and `fp8_dyn` slowest — an exact inversion of the bulk-throughput column. Two successive fixes failed. Direct instrumentation finally established the cause. On 650M/A100 with a 40-residue wild type over 11 masked positions:

Table 18: Compilation cost inside the timed region. A single forward pass at the DMS input shape costs 15 s to autotune and 28 ms to execute, so a timed region containing compilation ranks configurations by their number of Triton kernel candidates rather than by their speed.

Measurement	Time
eager, one forward at shape $(11, 42)$	27.9 ms
eager, full `masked_marginals` pass	0.03 s
compiled, one forward at shape $(11, 42)$	**15,138.1 ms**

DMS scoring uses a much smaller input shape than the bulk-throughput batches, and under `max-autotune` each new shape costs roughly 15 s to autotune. At this batch size that cost recurs rather than amortizing. The timed region was therefore measuring *autotune cost*, and ranking configurations by their number of Triton kernel candidates — `fp32` had the fewest and appeared fastest, `fp8_dyn` had 98 and appeared slowest. The fix was to score DMS against the uncompiled module.

This has a consequence beyond the benchmark: for short wild-type sequences with few mutated positions, `max-autotune` is actively counterproductive in production, not merely in measurement.

The same class of defect appeared earlier in the bulk-throughput path, where warming up on only a prefix of the batches left recompilation for unseen length buckets inside the timed region.

5.2 Peak memory attributed to the wrong model

Peak memory during DMS scoring was reported as follows, with the activation overhead implied by subtracting weights:

Table 19: The peak-memory artifact. Each configurations measured peak contains the previous configurations weights, which surfaces as bf16 apparently needing more peak memory than fp32.

Config	Weights (GB)	Peak (GB)	Implied overhead
`fp32`	2.49	2.56	+0.07
`bf16`	1.21	3.74	+2.53 ≈ fp32's weights
`int8_wo`	0.61	1.86	+1.25 ≈ bf16's weights
`int8_dyn`	0.61	1.26	+0.65 ≈ int8_wo's weights

Each configuration's peak included the *previous* configuration's weights, producing the visible absurdity of bf16 requiring more peak memory than fp32. The cleanup helper deleted only its own local reference to the model; the caller retained a binding. Because `nn.Module` graphs contain reference cycles, dropping the last reference does not free them by reference counting alone — they persist as cyclic garbage until a collection runs. Inserting an explicit `gc.collect()` before each peak-memory reset corrected it; overhead became a consistent 0.04–0.07 GB.

The bulk-throughput memory column was verified unaffected: bf16's measured overhead there was +7.34 GB, which cannot contain fp32's 10.72 GB of weights.

5.3 Silent argument leakage in the job script

A batch script used `shift 3 || true`. When fewer than three positional arguments are supplied, `shift 3` fails, `|| true` suppresses the failure, and the arguments remain in `"$@"` — where they were appended to the Python invocation as stray arguments. Relying on a default for the third argument was sufficient to trigger it.

5.4 Common thread

Three of the four defects shared a signature: a cost that belonged outside the measurement leaked inside it, and the resulting ranking was plausible enough to be believed. In each case the tell was a result that inverted or contradicted a neighbouring measurement. We would emphasise that a benchmark producing a *clean, monotonic, plausible* table is not thereby correct; the memory bug produced exactly such a table for several runs.

6 Discussion

6.1 The two workloads want opposite configurations

This is the principal practical finding.

Bulk embedding extraction is compute-bound: large batches, large GEMMs. Quantization pays. `fp8_dyn` with `max-autotune` wins on all three axes — 1.15× bf16 throughput, weights 5.29 → 2.65 GB, peak 12.63 → 7.24 GB, with accuracy indistinguishable from bf16.

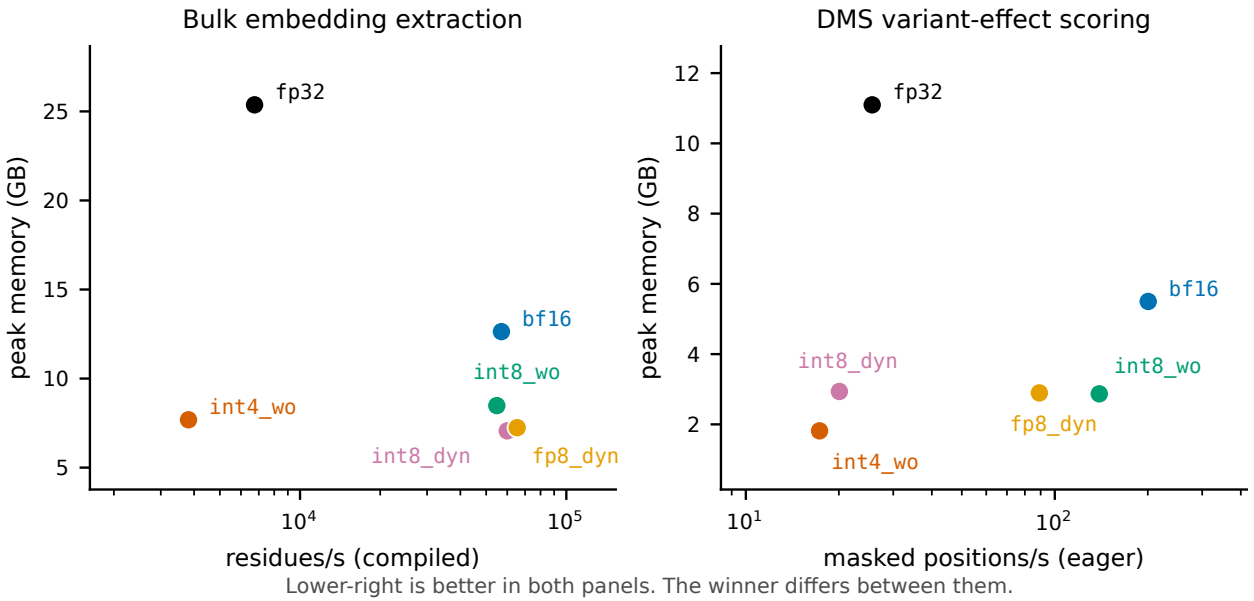

Figure 4: Speed–memory frontiers for the two workloads, ESM2-3B on one H200; lower-right is better in both panels. *Left:* bulk embedding extraction under `max-autotune`, where `fp8_dyn` is fastest and near-smallest. *Right:* DMS scoring over the full 201-assay benchmark in eager mode, where the same configuration is 2.2× slower than `bf16` and the ranking is substantially reordered. No single configuration is preferred by both.

DMS variant-effect scoring is latency-bound: the masked batch is small, no GEMM is large enough for low precision to help, and per-linear quantize/dequantize overhead is pure loss. bf16 in eager mode is fastest at every problem size, and `max-autotune` is actively harmful.

Applying the embedding-optimized configuration to DMS scoring makes it up to an order of magnitude slower. Because both workloads commonly appear in the same pipeline, this is a live risk rather than a theoretical one. Figure 4 shows the two frontiers side by side.

This conflict is bounded by scale, and we would not have known that from one model. The mechanism is that `fp8_dyn`'s per-call quantization overhead is fixed while the useful work grows with hidden size, so the penalty shrinks as the model grows: 0.30× bf16 DMS speed at 650M, 0.44× at 3B, 0.97× at 15B (Table 15). By 15B the DMS batch is large enough in the hidden dimension to be compute-bound despite being narrow in the batch dimension, and `fp8_dyn` becomes the right choice for *both* workloads at half the memory. The general lesson is not that DMS is latency-bound — it is that whether a workload is latency-bound is a property of the model and the workload together, not of the workload alone.

6.2 On the interpretation of fidelity metrics

The conventional validation practice — quantize, measure drift against the full-precision model, accept if drift is small — is sound as a *risk bound*. Our data support the bound at scale: over 3015 assay/configuration pairs, fidelity loss correlates with the magnitude of ground-truth change at $r = 0.74$, 0.81 and 0.56 at 650M, 3B and 15B, consistent with the 0.87 first estimated from 15 pairs.

It is not sound as a *quality ranking*. A configuration with larger drift is not thereby less accurate against reality; it is merely less predictable. The signed correlation is +0.10, −0.58 and −0.40 at 650M, 3B and 15B — it does not hold its sign across model scales.

A tempting misreading of our BLAT result is that quantization acts as beneficial regularization. We do not endorse this, and the full benchmark settles it: `int4_wo` is significantly *negative* at 650M,

significantly *positive* at 3B, and null at 15B. Two scales would have permitted the reading that sensitivity falls with model size; the third removes it. A configuration cannot be genuinely more accurate than the reference from which it is derived in any way one could rely on in advance.

6.3 Averages conceal the failure that matters

The benchmark-mean result is that no configuration changes mean ground-truth correlation by more than 0.007. Read alone, that licenses any configuration on the list. It should not.

`int8_dyn` at 3B is statistically indistinguishable from fp32 on the mean ($p = 0.34$) and takes one assay from $\rho = 0.591$ to 0.223. A practitioner does not deploy against the mean of 201 assays; they deploy against one protein. The benchmark mean answers whether a configuration is safe *on average over targets*, which is a claim about a population, while the operative question is whether it is safe on a specific target.

The practical consequence is that the tail statistics — worst-case single-assay drift, and the count of assays exceeding a tolerance — are the numbers to select on, and they separate the configurations far more sharply than the means do. On that criterion `bf16` and `int8_wo` are safe (no assay beyond 0.018 across 402 assay/model pairs), `fp8_dyn` is nearly so (0.034), and `int8_dyn` and `int4_wo` are not.

6.4 Recommendations

Table 20: Configuration recommendations by situation. The first two rows differ because the two workloads sit in different bottleneck regimes, not because of any accuracy difference.

Workload	Scale	Recommendation
Bulk embedding extraction	any (H200)	`fp8_dyn` + `max-autotune-no-cudagraphs`
Bulk embedding extraction	any (A100)	`int8_wo`; `int8_dyn` costs real accuracy
DMS / variant effect	$\leq$ 3B	`bf16`, eager, no compile
DMS / variant effect	15B	`fp8_dyn` — matches bf16 speed at half the memory
DMS under memory pressure	$\leq$ 3B	`int8_wo` ($\sim$70% of bf16 speed)
Minimum memory footprint	any	`int4_wo`, only after confirming drift on *your* target; worst observed -0.20
QLoRA fine-tuning base	any	`int4_wo`
V100 / sm_70 hardware	any	Quantization buys memory only, never speed

Two non-quantization measures outweigh most of the above. Moving from fp32 to bf16 is worth 8.4× on bulk extraction and 6.2× over the full DMS benchmark; length-bucketed batching eliminates 25% of wasted computation. Both should be applied before any quantization is considered.

A third, proposed in an earlier draft of this report, does not survive the full benchmark: the 650M checkpoint beat 3B by 0.14 Spearman on BLAT, which suggested model selection dominated precision selection. Over 201 assays the two are indistinguishable (0.4459 vs 0.4398, 95% CI $[-0.0061, +0.0185]$, $p = 0.35$, with 650M ahead on 110 of 201). What remains true is narrower and still useful: the *uncertainty* on model choice (± 0.012) exceeds every quantization effect measured here (≤ 0.007). Precision is the settled variable; checkpoint choice is not, and is worth benchmarking on a representative assay.

6.5 A selection protocol

Our results imply a procedure rather than a ranking, because the quantity that separates configurations — worst-case single-assay drift — is cheap to measure on a target and cannot be inferred from published means. Algorithm 3 states it. The essential point is line 4: the screen runs against the fp32 model on the user's own sequences and needs no experimental labels, so it costs one extra scoring pass and no wet-lab work.

Algorithm 3 Choosing a precision configuration for a variant-effect workload. The screen requires no ground-truth labels, because Section 6.2 establishes that drift from fp32 is a valid bound on how far the answer can move even though it does not indicate direction.

Require: target protein(s) S, tolerance τ on Spearman drift, memory budget m

1: score S under `bf16`; **if** it fits in m, **return** `bf16` ▷ fastest at every problem size we measured
2: $\mathcal{C} \leftarrow \langle$`int8_wo`, `fp8_dyn`$\rangle$ ▷ ordered by measured worst-case drift, not by mean
3: **for** $c \in \mathcal{C}$ with weight footprint $\leq m$ **do**
4: $\quad \rho_c \leftarrow \mathrm{Spearman}\big(\sigma_c(V_S),\ \sigma_{\mathrm{fp32}}(V_S)\big)$ ▷ one scoring pass; no labels needed
5: $\quad$ **if** $1 - \rho_c \leq \tau$ **then return** c
6: **end for**
7: **if** m still unmet: use `int4_wo` **only** after confirming $1 - \rho_c \leq \tau$ on *this* target ▷ worst observed single-assay collapse: -0.20
8: **else** reduce m by reducing model scale before reducing precision

We deliberately do not recommend selecting on published benchmark means. On the 201-assay mean, `int8_dyn` and `bf16` are separated by 0.002; on worst-case single-assay behaviour they are separated by 0.36. The mean is the statistic most likely to be quoted and the least useful for this decision.

6.6 Computational cost

The full benchmark costs 1.4 GPU-hours at 650M, 3.1 at 3B and 11.7 at 15B on a single H200 — 40,775 masked forward passes per configuration, six configurations, three scales, 16.2 GPU-hours in total. Cost is dominated by the two configurations we recommend against: `int4_wo` alone is 5.5 of the 11.7 hours at 15B. Restricting to fp32 plus the three viable configurations would cost 4.7 hours at 15B and 6.4 overall.

This is small enough that evaluating on the complete benchmark rather than a subset is a matter of choosing to, not of affording to. The three-assay evaluation this study began with saved roughly two GPU-hours and produced two conclusions that the full benchmark reversed; running one scale rather than three would have saved a further thirteen and left a third.

7 Limitations

- **We still cannot predict which assays behave which way.** 201 assays establish the population behaviour of each configuration and the size of its tail. They do not identify in advance which target will be the one that collapses. The only reliable procedure remains scoring a candidate configuration against fp32 on the actual target.
- **Three scales are enough to refute a trend, not to establish one.** We can say the `int4_wo` effect has no monotone dependence on model size, and that the workload conflict

dissolves somewhere between 3B and 15B. We cannot say where that boundary lies, nor whether the `fp8_dyn` crossover continues past 15B, because ESM-2 has no larger checkpoint.

- **15B is less numerically stable even at bf16.** Its worst-case `bf16` drift is -0.0323 against -0.0072 at 3B (Table 11). We did not isolate the cause; a fp16-versus-bf16 comparison and a per-layer error analysis would be the next step.
- **The 16 longest assays are excluded.** Assays beyond 1022 residues exceed ESM-2's context and were skipped rather than truncated, since a truncation window is a scoring-protocol change that would confound the comparison. Configuration behaviour above $L = 934$ is therefore unmeasured.
- **Multiple comparisons are uncorrected.** Five configurations are tested against fp32 per model. Both significant results survive a Bonferroni threshold of $0.05/5 = 0.01$, but `int4_wo` at 650M sits exactly on it ($p = 0.010$).
- **Bootstrap resolution.** At 2000 resamples the smallest reportable two-sided p is 0.001; values at that floor should be read as $p < 0.002$ rather than as point estimates.
- **Low-signal assays resolve nothing individually.** Where $\rho_{\text{expt}} \approx 0.3$ the model's own error dominates and configuration differences fall below the noise floor; such assays contribute to the benchmark mean but carry little information alone.
- **Sequence-length coverage.** Bulk-throughput measurements used sequences of 512–1024 residues. Beyond $L \approx 2000$ the $O(L^2)$ attention term dominates peak memory and quantization's share of the memory benefit shrinks correspondingly. A representative FASTA from the target workload would settle this.
- **Single random seed** for quantization; no ensemble over calibration variation.
- **The QLoRA track is unbuilt.** `int4_wo` at 1.61 GB is the natural base, and fine-tuning is the one setting where quantization buys capability rather than efficiency — full 3B fine-tuning requires roughly 45 GB of optimizer state alone.

8 Conclusions

What to run. Two workloads, and the answer depends on model scale.

- **Bulk embedding extraction: `fp8_dyn` with `max-autotune`.** Best on all three axes at once at 3B — 1.15× bf16 throughput, peak memory 12.63 → 7.24 GB, accuracy indistinguishable from bf16. Needs FP8 hardware (sm_90); on A100, `int8_wo`.
- **Variant-effect scoring up to 3B: `bf16`, eager, uncompiled.** Fastest at every problem size from $L = 37$ to 934 and 6.2× faster than fp32 over the full benchmark. Quantization is pure overhead here and compilation is actively harmful.
- **Variant-effect scoring at 15B: `fp8_dyn`.** It matches bf16 speed (0.97×) at half the peak memory (14.6 vs 28.6 GB) with no measurable accuracy cost. The recommendation inverts with scale.
- **Avoid `int8_dyn` and `int4_wo` for variant-effect work at any scale.** They are not trade-offs: both are slower than the best option in this regime *and* own the two worst tails we measured, -0.37 and -0.20.

What the evidence says about evidence. Four findings generalise past ESM-2, and each contradicts a common practice.

1. **Select on worst-case drift, not on benchmark means.** At no scale does any configuration move mean correlation by more than 0.007 — the statistic most likely to be published, and the one least able to distinguish these configurations. `int8_dyn` passes the mean test at 3B ($p = 0.34$) and takes one assay from $\rho = 0.591$ to 0.223. On means, `int8_dyn` and `bf16` differ by 0.002; on worst case, by 0.36 — a factor of 180.
2. **Drift from fp32 is a valid safety certificate and an invalid quality ranking.** Over 3015 assay/configuration pairs it predicts the *magnitude* of ground-truth change ($r = 0.56$–0.81) but not its *direction*, whose correlation changes sign across model scales. A configuration close to fp32 is safe; one far from fp32 is merely unpredictable, not worse. `bf16` and `int8_wo` never exceed 0.018 Spearman on any of 402 assay/model pairs up to 3B and are safe on that basis alone — cheap to verify on a target, with no experimental labels required (Algorithm 3).
3. **Conclusions are bounded by model scale, not only by sample size.** The workload conflict that motivates half this paper holds at 650M and 3B and dissolves at 15B. The `int4_wo` effect is significantly negative, significantly positive, and null at the three scales in ascending order — not a trend, so nothing to extrapolate. Even the stratification by MSA depth replicates at two scales and breaks at the third. A result measured at one scale should be reported as a result at that scale.
4. **Evidence scale is not a matter of rigour but of conclusions.** This study reached four successive verdicts. A synthetic mutational scan ranked configurations confidently and wrongly. Three real ProteinGym assays overturned it and produced two further generalisations — that quantization does not systematically degrade accuracy, and that checkpoint choice dominates precision choice — both of which the full 201-assay benchmark reversed. That benchmark at two scales then suggested quantization sensitivity grows with model size, which the third scale removed. Every intermediate conclusion was statistically sound on its own data and looked well-supported. The full three-scale benchmark costs 16.2 GPU-hours.

The general claim. A benchmark mean answers whether a configuration is safe *on average over targets*. Deployment asks whether it is safe on *one* target. These questions have different answers here, and the gap between them is not a detail — it is the difference between 0.002 and 0.36. Wherever a benchmark aggregates over heterogeneous instances and the user faces one instance, we expect the same gap, and we would report the tail alongside the mean as a matter of course.

9 Data and Code Availability

All assay data is redistributed from ProteinGym [11], release `v1`, obtained from the `OATML-Markslab/ProteinGym_v1` dataset repository. No new experimental measurements were produced.

The following are released with this work:

- Benchmark harness, scoring driver, and analysis code, including the extraction script that validates every variant's wild-type residue against `target_seq` before writing.
- **Per-variant predictions** for all 18 model/configuration combinations: 2,413,913 scores each, in the canonical assay row order, so that any alternative statistic can be recomputed without access to a GPU.

- Per-assay statistics (1206 rows per model scale: accuracy, wall-clock, peak memory, and fidelity for every assay/configuration pair) and the aggregated summaries reported in Section 4.

Code, per-assay statistics, and both documents are available at `https://github.com/qshao/esm2-quantization`.

The per-variant predictions are excluded from that repository on size grounds (≈90 MB compressed) and are regenerated by the batch script it contains; the derived per-assay table, from which every result in this paper is computed, is included in full.

References

[1] J. Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ASPLOS*, 2024.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.

[3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022.

[4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *NeurIPS*, 2023.

[5] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[6] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

[7] Z. Lin et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[8] J. Lin et al. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *MLSys*, 2024.

[9] J. Meier, R. Rao, R. Verkuil, J. Liu, T. Sercu, and A. Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. In *NeurIPS*, 2021.

[10] P. Micikevicius et al. FP8 formats for deep learning. *arXiv:2209.05433*, 2022.

[11] P. Notin et al. ProteinGym: Large-scale benchmarks for protein fitness prediction and design. In *NeurIPS Datasets and Benchmarks*, 2023.

[12] A. Rives et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *PNAS*, 118(15):e2016239118, 2021.

[13] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.

[14] torchao maintainers. torchao: PyTorch-native training-to-serving model optimization. `https://github.com/pytorch/ao`, 2024.

[15] K. Tsuboyama et al. Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620:434–444, 2023.

A Reproduction

```
# Environment (handles glibc/GCC/cache-quota constraints)
source env/activate.sh
```

```
# Fetch assay data; verifies variant indexing against target_seq
python src/fetch_proteingym.py IF1_ECOLI_Kelsic_2016 \
    BLAT_ECOLX_Stiffler_2015 HSP82_YEAST_Flynn_2019

# Bulk throughput + fidelity matrix (compiled)
sbatch -p H8V141_SAP112M2000_L --mem=200G slurm/run_matrix.sbatch \
    3B fp32,bf16,int8_wo,int8_dyn,fp8_dyn,int4_wo --compile

# Real-assay DMS: accuracy, speed, memory (uncompiled by design)
sbatch -p H8V141_SAP112M2000_L --mem=200G slurm/run_dms.sbatch \
    3B fp32,bf16,int8_wo,int8_dyn,fp8_dyn,int4_wo

# Significance of ground-truth shifts (single assay, variant bootstrap)
python src/analyze_dms.py results/dms_3B_*_<jobid>_scores.json

# --- Full benchmark ------------------------------------------------
# All 217 assays; validates every variant's WT and refuses to write on
# mismatch.  ~110 MB of parquet, cached in data/proteingym_v1/_raw
python src/fetch_proteingym_v1.py

# One array task per config; each loads its model once and scores all
# 201 assays.  Resumable: a resubmission skips assays already written.
sbatch --array=0-5 -p H8V141_SAP112M2000_L --mem=200G \
    slurm/run_proteingym.sbatch 3B
sbatch --array=0-5 -p H8V141_SAP112M2000_L --mem=200G \
    slurm/run_proteingym.sbatch 650M

# Benchmark-wide accuracy / speed / RAM + protein-clustered bootstrap
python src/aggregate_proteingym.py --model 3B
python src/aggregate_proteingym.py --model 650M
```

Result artifacts referenced in this report:

- Throughput matrices (SLURM jobs 5393462, 5393520, 5393524, 5393554): `results/matrix_3B_<jobid>.json`
- ProteinGym, 3B, with per-variant scores (job 5394386): `results/dms_3B_<assay>_5394386.json`
- ProteinGym, 650M (job 5394316): `results/dms_650M_<assay>_5394316.json`
- Full benchmark, per-variant scores, 3B (array 5395036) and 650M (array 5395037): `results/proteingym/<model>_<config>.jsonl.gz`
- Full-benchmark analysis: `results/proteingym/summary_<model>.json` and `summary_<model>_per_assay.csv` (1206 rows: one per assay/configuration)